AI Course Development Pipeline — Architecture Blueprint

Project Overview

An AI system that decomposes course creation into a visible, interactive agent graph. Specialized sub-agents handle research, drafting, and review in parallel — with explicit autonomy levels, human-in-the-loop checkpoints, and a transparent decision log. The system demonstrates how AI can reason about its own boundaries and defer to humans at the right moments.

Tech Stack

Layer	Technology
Frontend	React + React Flow (interactive node graph), Tailwind CSS
Backend	Python, FastAPI
LLM	Anthropic Claude API (claude-sonnet-4-20250514)
State/Storage	SQLite (decision log, catalog), JSON (task graph state)
Package Management	npm (frontend), pip (backend)

File Structure

```
course-pipeline/
├── backend/
│   ├── main.py              # FastAPI app entry point
│   ├── orchestrator.py      # Task graph generation and execution engine
│   ├── agents/
│   │   ├── base.py          # Base agent class (shared logic, logging)
│   │   ├── research.py      # Core Topic Research agent
│   │   ├── audience.py      # Audience Analysis agent
│   │   ├── catalog.py       # Catalog Search agent
│   │   ├── outline.py       # Outline Generator agent
│   │   ├── script.py        # Script Drafter agent
│   │   ├── review_technical.py  # Technical Fidelity Reviewer
│   │   ├── review_pedagogy.py   # Pedagogy Reviewer
│   │   ├── review_business.py   # Business Alignment Reviewer
```

```
│   │   └── conflict.py        # Conflict Resolution agent
│   ├── models.py              # Pydantic models for all data structures
│   ├── database.py            # SQLite setup, decision log, catalog
│   ├── dummy_catalog.py       # Seed data for dummy course catalog
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.jsx            # Root component
│   │   ├── components/
│   │   │   ├── BriefWizard.jsx  # Multi-step structured intake form
│   │   │   ├── PipelineGraph.jsx # React Flow graph visualization
│   │   │   ├── NodeDetail.jsx   # Side panel showing selected node's details
│   │   │   ├── DecisionLog.jsx  # Scrollable audit log
│   │   │   ├── ReviewPanel.jsx  # Three-reviewer comparison view
│   │   │   ├── ConflictCard.jsx # Disagreement display with human decision UI
│   │   │   └── ChatPanel.jsx    # Optional free-form chat with any node
│   │   ├── hooks/
│   │   │   └── usePipeline.js   # WebSocket/polling hook for live graph updates
│   │   └── utils/
│   │       └── graphLayout.js   # Node positioning logic for React Flow
│   ├── package.json
│   └── index.html
├── README.md                  # Project narrative (part of submission)
└── .env                       # API keys (not committed)
```

Data Structures

Course Brief (user input)

```
json
```

```
{
  "title": "string",
  "topic_area": "enum: ML | Data Science | Ethics | Programming | Applied AI | Other",
  "description": "string (max 500 chars)",
  "audience": "enum: Executives | Technical PMs | Junior Devs | Senior Engineers | Non-technical",
  "prerequisites": ["string"],
  "class_size": "enum: small (<15) | medium (15-40) | large (40+)",
  "duration": "enum: 1hr talk | half-day | full-day | multi-day",
  "format": "enum: in-person | virtual | self-paced",
  "balance": "float 0-1 (0 = fully conceptual, 1 = fully hands-on)",
  "learning_objectives": ["string (max 3)"],
  "outcome_description": "string"
}
```

Task Graph Node

```json
{
  "id": "string (unique)",
  "type": "enum: orchestrator | research | audience | catalog | outline | script | review_technical | review_pedagogy | review_bus
  "label": "string (display name)",
  "status": "enum: pending | running | completed | blocked | awaiting_human | error",
  "autonomy": "enum: full | draft | advisory | escalate | refuse",
  "input_data": {},
  "output_data": {},
  "dependencies": ["node_id"],
  "blocked_by": "node_id | null",
  "reasoning": "string (why this node exists, logged for decision trail)",
  "started_at": "timestamp | null",
  "completed_at": "timestamp | null"
}
```

Review Output (per reviewer)

```json
```

```
{
  "reviewer_type": "enum: technical | pedagogy | business",
  "module_id": "string",
  "verdict": "enum: approve | flag | reject",
  "confidence": "float 0-1",
  "findings": [
    {
      "severity": "enum: info | warning | critical",
      "category": "string",
      "description": "string",
      "suggestion": "string"
    }
  ],
  "reasoning": "string"
}
```

Conflict Report

```
{
  "module_id": "string",
  "agreements": [
    {
      "topic": "string",
      "shared_recommendation": "string"
    }
  ],
  "disagreements": [
    {
      "topic": "string",
      "positions": {
        "technical": "string",
        "pedagogy": "string",
        "business": "string"
      },
      "conflict_type": "enum: factual | subjective | priority",
      "ai_assessment": "string (why the AI cannot resolve this)",
      "human_options": ["string (possible decisions for the human)"]
    }
  ]
}
```

Decision Log Entry

```json
{
  "id": "auto-increment",
  "timestamp": "ISO datetime",
  "node_id": "string",
  "node_type": "string",
  "action": "string (what the agent did)",
  "reasoning": "string (why)",
  "confidence": "float 0-1",
  "autonomy_level": "string",
  "human_override": "boolean",
  "human_decision": "string | null"
}
```

Agent Specifications

Layer 1: Orchestrator

- **Trigger:** User submits completed course brief form.
- **Input:** Course brief JSON.
- **Process:** Analyzes the brief and generates the full task graph — determines which nodes are needed, their dependencies, and which can run in parallel. For multi-module courses, dynamically creates N script drafter nodes and N×3 reviewer nodes.
- **Output:** Complete task graph (list of nodes with dependencies). Presented to user for approval before execution begins.
- **Autonomy:** FULL — but user sees and approves the plan before anything runs.
- **Human checkpoint:** User reviews the proposed graph and clicks "Start Pipeline."

Layer 2a: Core Topic Research

- **Trigger:** Pipeline starts (no dependencies).
- **Input:** topic_area, description, learning_objectives from brief.
- **Process:** Generates structured topic breakdown — key concepts, subtopics, prerequisite knowledge chain, suggested depth per subtopic given duration.
- **Output:** Topic map with concepts ranked by importance.
- **Autonomy:** FULL.

- **Failure mode:** If topic is too broad for the duration, flags this as a warning on the node.

Layer 2b: Audience Analysis

- **Trigger:** Pipeline starts (no dependencies, runs parallel with 2a and 2c).
- **Input:** audience, prerequisites, class_size, balance from brief.
- **Process:** Generates audience profile — assumed knowledge, likely pain points, preferred learning modalities, attention span estimates, recommended example types.
- **Output:** Audience profile document.
- **Autonomy:** RECOMMENDS — output is presented to user with "Does this match your understanding of this audience?" prompt. User can edit/confirm.
- **Human checkpoint:** User confirms or adjusts audience assumptions.

Layer 2c: Catalog Search

- **Trigger:** Pipeline starts (no dependencies, runs parallel with 2a and 2b).
- **Input:** topic_area, description, learning_objectives + full course catalog from DB.
- **Process:** Searches existing catalog for: (a) direct topic overlap, (b) reusable modules (e.g., an ethics section inside a biology course), (c) potential full duplicates.
- **Output:** Overlap report with specific module-level matches and reuse recommendations.
- **Autonomy:** FULL — but if overlap score > 70%, it BLOCKS the outline generator and escalates: "Significant overlap found with [course]. Human must decide whether to proceed, merge, or differentiate."
- **Blocking behavior:** Sets outline node status to "blocked" with reason.

Layer 3: Outline Generator

- **Trigger:** All Layer 2 nodes completed (or confirmed by human). Catalog block resolved if triggered.
- **Input:** Topic map + audience profile + overlap report.
- **Process:** Generates structured course outline — modules, learning objectives per module, time allocation, activity types, prerequisite flow between modules.
- **Output:** Structured outline (list of modules with metadata).
- **Autonomy:** DRAFT — presented to user for approval. User can reorder modules, edit objectives, remove/add modules.
- **Human checkpoint:** User approves or edits the outline before script drafting begins.
- **Pedagogical checks (built-in):**
    - Prerequisites taught before they're needed

- Cognitive load per module (not too many new concepts)
- Balance between conceptual and hands-on matches the user's slider
- No orphan modules (modules that don't connect to learning objectives)

Layer 4: Script Drafters (×N, one per module)

- **Trigger:** Outline approved by human. All script nodes run in parallel.
- **Input:** Single module from the approved outline + audience profile + topic map.
- **Process:** Generates a lesson script including: speaker notes/talking points, examples, analogies appropriate for the audience, activities/exercises, transition notes (how this connects to previous/next module).
- **Output:** Script document for one module.
- **Autonomy:** DRAFT — all scripts are drafts by definition.

Layer 5a: Technical Fidelity Reviewer

- **Trigger:** Script draft completed for a module. Runs in parallel with 5b and 5c.
- **Input:** Script draft + topic map (for reference accuracy).
- **Process:** Reviews for: factual accuracy, appropriate complexity level, correct terminology, whether examples are technically sound, whether simplifications introduce misconceptions.
- **Output:** Review report (findings with severity + suggestions).
- **Autonomy:** ADVISORY — cannot modify the script, only flag issues.

Layer 5b: Pedagogy Reviewer

- **Trigger:** Script draft completed for a module. Runs in parallel with 5a and 5c.
- **Input:** Script draft + audience profile + outline (for flow context).
- **Process:** Reviews for: cognitive load, engagement patterns (too much lecture without interaction?), prerequisite flow within the module, clarity of explanations, activity quality, transition smoothness.
- **Output:** Review report (findings with severity + suggestions).
- **Autonomy:** ADVISORY.

Layer 5c: Business Alignment Reviewer

- **Trigger:** Script draft completed for a module. Runs in parallel with 5a and 5b.
- **Input:** Script draft + catalog overlap report + original brief.

- **Process:** Reviews for: alignment with stated learning objectives, ROI (is this the right depth for the time investment?), overlap with existing catalog (are we rebuilding something that exists?), audience fit, scope creep from the original brief.
- **Output:** Review report (findings with severity + suggestions).
- **Autonomy:** ADVISORY.

Layer 6: Conflict Resolver

- **Trigger:** All three reviewers completed for a module.
- **Input:** Three review reports.
- **Process:**
  1. Identifies agreements — where all reviewers align, merge into unified recommendation.
  2. Identifies disagreements — where reviewers conflict.
  3. Classifies each disagreement:
     - **Factual:** One reviewer is objectively right → AI resolves (rare).
     - **Priority:** Reviewers agree on fact but disagree on importance → AI surfaces trade-off, human decides.
     - **Subjective:** Different values/perspectives → AI explicitly refuses to resolve, presents both sides.
  4. For each unresolved disagreement, generates clear human-facing options.
- **Output:** Conflict report with agreements, disagreements, and decision prompts.
- **Autonomy:** ESCALATE — can merge agreements, cannot resolve disagreements. Explicitly states why it can't decide.
- **Human checkpoint:** User sees disagreements and makes final calls.

Dummy Course Catalog (seed data)

Six fictional courses, loosely inspired by a generic AI education organization:

#	Title	Duration	Audience	Topics/Modules
1	Introduction to Machine Learning for Business Leaders	1-day workshop	Executives	ML fundamentals, business applications, build-vs-buy, case studies, ROI of ML projects
2	Responsible AI: Ethics and Governance	Half-day workshop	Mixed	Bias and fairness, transparency, accountability frameworks, regulatory landscape, ethics review processes
3	Deep Learning Fundamentals	3-day course	Junior Devs / Senior Engineers	Neural networks, CNNs, RNNs, transformers, training pipelines, optimization, practical exercises
4	AI for Healthcare Applications	1-day workshop	Technical PMs / Senior Engineers	Medical imaging, NLP for clinical notes, ethics in healthcare AI, regulatory requirements (Health Canada, FDA), case studies
5	Building with Large Language Models	Full-day hands-on	Junior Devs / Senior Engineers	Prompt engineering, RAG, fine-tuning, evaluation, deployment, cost management, responsible use
6	Data Literacy for Non-Technical Teams	2-hour talk	Non-technical	What is data, reading charts, basic statistics, data-driven decisions, common pitfalls, hands-on spreadsheet exercise

Key overlaps built into the catalog (for the Catalog Search agent to find):

- Ethics appears in courses 2, 4, and partially in 1 and 5
- ML fundamentals overlap between courses 1 and 3
- Hands-on LLM content overlaps between courses 3 and 5
- Healthcare regulatory content in course 4 partially overlaps with governance in course 2
- Course 6 has prerequisite-level content for courses 1 and 4

UI Flow

Screen 1: Brief Wizard (structured intake form)

Multi-step form, one section at a time, with progress indicator.

- **Step 1 — Topic:** Title field, topic area dropdown, description textarea (with character counter and placeholder example).

- **Step 2 — Audience:** Audience dropdown, prerequisite multi-select checklist, class size radio buttons.
- **Step 3 — Format:** Duration dropdown, delivery format radio, conceptual↔hands-on slider.
- **Step 4 — Goals:** Three learning objective fields (with placeholder examples), outcome textarea.
- **Step 5 — Review & Submit:** Summary card of all inputs. User confirms and clicks "Generate Pipeline."

Design notes:

- Each step has helper text explaining why this information matters
- Back/forward navigation between steps
- Cannot submit without all required fields
- Clean, minimal design — no clutter

Screen 2: Pipeline View (main application screen)

Layout: Three-panel layout.

Left panel (narrow): Brief summary card (collapsible) + Decision Log (scrollable, newest first). Each log entry shows: timestamp, agent name, action, autonomy level, and any human overrides.

Center panel (main): React Flow graph visualization.

- Nodes are colored by status: gray (pending), blue (running), green (completed), yellow (awaiting human), red (blocked/error)
- Edges show data flow direction with animated dots when data is being passed
- Clicking a node selects it and opens its details in the right panel
- The graph auto-layouts to show the parallel/convergence structure clearly

Right panel: Detail view for the selected node.

- Shows: node name, type, autonomy level, status
- Input data (collapsible)
- Output data / results (collapsible)
- Reasoning / decision explanation
- If status is "awaiting_human": shows the human decision UI (approve/edit/reject buttons, or conflict resolution options)
- If status is "blocked": shows why and what needs to happen

Human Checkpoint Interactions

When a node reaches "awaiting_human" status:

- The node pulses yellow in the graph
- The right panel shows the output + a decision interface
- For **Audience Analysis**: "Does this match your understanding?" + editable fields + Confirm/Edit buttons
- For **Outline**: Full outline display with drag-to-reorder modules, edit buttons per module, Approve/Request Changes buttons
- For **Conflict Resolution**: Cards showing each disagreement with the three reviewer positions, AI's explanation of why it can't decide, and clickable options for the human's decision

Chat Panel (secondary, toggleable)

A slide-out chat drawer (bottom or right edge). The user can ask any node a follow-up question: "Why did you flag this?" "Can you make module 2 more hands-on?" This is free-form, but scoped to the selected node's context.

API Endpoints

```
POST   /api/brief              — Submit course brief, returns pipeline_id
GET    /api/pipeline/{id}      — Get full pipeline state (all nodes, statuses)
POST   /api/pipeline/{id}/start — Start pipeline execution
GET    /api/node/{id}          — Get single node detail + output
POST   /api/node/{id}/approve  — Human approves a node's output
POST   /api/node/{id}/edit     — Human edits a node's output and approves
POST   /api/node/{id}/decide   — Human makes a decision on a conflict
GET    /api/log/{pipeline_id}  — Get decision log entries
POST   /api/chat/{node_id}     — Send a free-form message to a node's agent
GET    /api/catalog            — Get dummy course catalog (for transparency)

WebSocket /ws/pipeline/{id}    — Live updates as nodes change status
```

Autonomy Framework Summary

This is the philosophical core of the project. Include this prominently in the README.

Level	Label	What the AI does	What the human does	Example
1	**FULL**	Acts, logs reasoning	Can audit after the fact	Core topic research
2	**DRAFT**	Produces output, presents it	Must approve/edit before it's used downstream	Outline, scripts
3	**RECOMMEND**	Proposes with rationale	Confirms or overrides	Audience assumptions
4	**ADVISORY**	Reviews, flags, suggests	Decides what to act on	All three reviewers
5	**ESCALATE**	Surfaces the problem + options	Makes the decision	Conflict resolution
6	**REFUSE**	Declines to act, explains why	Must handle it themselves	Unresolvable conflicts, out-of-scope requests

The key design principle: **autonomy decreases as subjectivity increases.** Factual tasks (research) get high autonomy. Judgment calls (is this the right tone for the audience?) get escalated to humans. The AI never pretends to make a subjective decision — it names the trade-off and steps back.

README Narrative Outline (for submission)

The README should tell the story of the project. Suggested structure:

1. **The Problem:** Knowledge-intensive organizations waste enormous time duplicating work. Course creation involves research, drafting, and review — each step has failure modes that AI can help with, but also judgment calls that AI should not make alone.
2. **The System:** An AI pipeline that decomposes course creation into specialized agents with explicit, tiered autonomy. The human stays in the loop at every decision point that involves subjectivity, values, or institutional knowledge.
3. **The Autonomy Framework:** (the table above, with explanation of the design philosophy)
4. **Architecture:** The graph, the agents, the data flow.
5. **What the AI does well:** Parallel research, draft generation, multi-perspective review, surfacing conflicts.
6. **What the AI explicitly does NOT do:** Resolve subjective disagreements, override human decisions, make assumptions about audience without confirmation, proceed when it detects significant catalog overlap.

7. **What I'd build next:** (shows vision beyond the prototype) — real document integration, learning from past human decisions to improve future suggestions, integration with actual course delivery platforms.

Implementation Priority Order

If time runs short, build in this order (each layer is a complete, demoable increment):

Priority 1 (MVP — must have):

- Brief wizard form
- Orchestrator that generates a static graph
- React Flow visualization with node statuses
- 2-3 agents actually running (research + outline + one reviewer)
- Decision log

Priority 2 (strong submission):

- All Layer 2 agents running in parallel
- Outline with human approval checkpoint
- All three reviewers running
- Conflict resolution with human decision UI

Priority 3 (impressive submission):

- Script drafting for multiple modules
- Full parallel execution with live status updates via WebSocket
- Chat panel for node interaction
- Polished visual design

Priority 4 (stretch):

- Animated graph transitions
- Export final course package
- Demo video / walkthrough